Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «Операционные системы»
За 1 семестр
Тема: «GCC. Процессы»
Вариант 7

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
№ зачётной книжки: (250886)
Проверил:
Давидюк Ю.И.

Брест 2026

Цель работы: Познакомиться с компилятором GCC; изучить порождение, завершение и замену процессов в операционной системе Linux.

Ход работы:

Вариант:

fork - `0 1 1 1 3 3 5 5`;

exec - `7`;

команда - `ps -u [user]`.

Задание:

Написать программу, которая будет реализовывать следующие функции:
• сразу после запуска получает и сообщает свой ID и ID родительского процесса;
• перед каждым выводом сообщения об ID процесса и родительского процесса эта информация получается заново;
• порождает процессы, формируя генеалогическое дерево согласно варианту, сообщая, что "процесс с ID таким-то породил процесс с таким-то ID";
• перед завершением процесса сообщить, что "процесс с таким-то ID и таким-то ID родителя завершает работу";
• один из процессов должен вместо себя запустить программу, указанную в варианте задания.

На основании выходной информации программы предыдущего пункта изобразить генеалогическое дерево процессов (с указанием идентификаторов процессов). Объяснить каждое выведенное сообщение и их порядок в предыдущем пункте.

Код программы:

```
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <pwd.h>

#define N 8
#define EXEC_NUM 7

static const int parent[N + 1] = {0, 0, 1, 1, 1, 3, 3, 5, 5};

static void say_start(int num)
{
    printf("Процесс №%d: ID = %d, ID родителя = %d\n",
           num, getpid(), getppid());
    fflush(stdout);
}

static void finish(int num)
{
    if (num == EXEC_NUM) {
        struct passwd *pw = getpwuid(getuid());
        const char *user = pw ? pw->pw_name : "root";

        printf("Процесс с ID %d и ID родителя %d "
               "завершает работу (запускает ps -u %s)\n",
               getpid(), getppid(), user);
        fflush(stdout);

        execlp("ps", "ps", "-u", user, (char *)NULL);
        perror("execlp");
        exit(1);
    }
    printf("Процесс с ID %d и ID родителя %d "
           "завершает работу\n", getpid(), getppid());
    fflush(stdout);
    exit(0);
}

static void run(int num)
{
    int i;
    pid_t pid;

    say_start(num);

    for (i = num + 1; i <= N; i++) {
        if (parent[i] != num)
            continue;

        fflush(stdout);
        pid = fork();
```

```
        if (pid == -1) {
            perror("fork");
            exit(1);
        } else if (pid == 0) {

            usleep(100000);
            run(i);
        } else {
            printf("Процесс с ID %d породил процесс с ID %d "
                   "(процесс №%d)\n", getpid(), pid, i);
            fflush(stdout);

            waitpid(pid, NULL, 0);
        }
    }

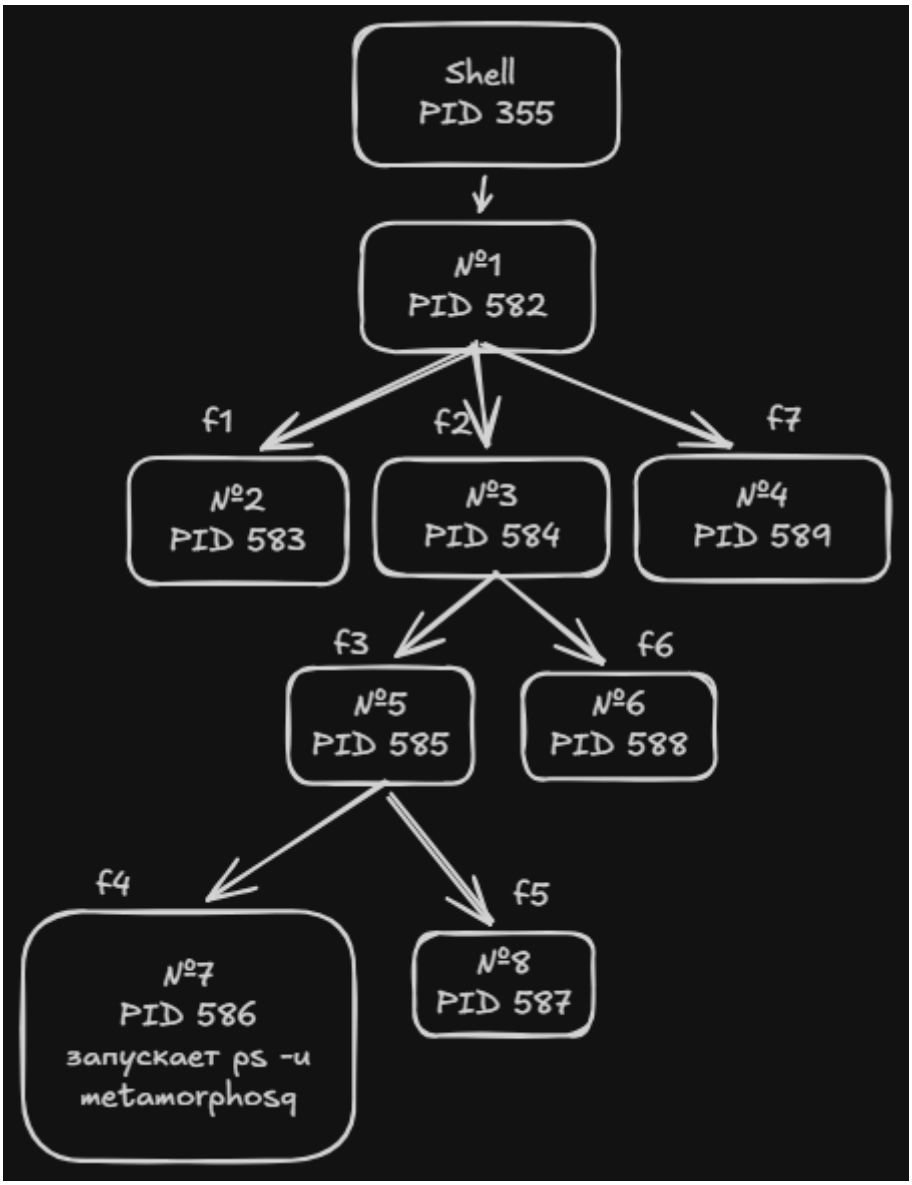
    finish(num);
}

int main(void)
{
    run(1);
    return 0;
}
```

3. Результат:

```
metamorphosq@ftshsm:~$ gcc proc.c -o proc
metamorphosq@ftshsm:~$ ./proc
Процесс №1: ID = 582, ID родителя = 355
Процесс с ID 582 породил процесс с ID 583 (процесс №2)
Процесс №2: ID = 583, ID родителя = 582
Процесс с ID 583 и ID родителя 582 завершает работу
Процесс с ID 582 породил процесс с ID 584 (процесс №3)
Процесс №3: ID = 584, ID родителя = 582
Процесс с ID 584 породил процесс с ID 585 (процесс №5)
Процесс №5: ID = 585, ID родителя = 584
Процесс с ID 585 породил процесс с ID 586 (процесс №7)
Процесс №7: ID = 586, ID родителя = 585
Процесс с ID 586 и ID родителя 585 завершает работу (запускает ps -u metamorphosq)
    PID TTY          TIME CMD
    355 pts/0    00:00:00 bash
    404 ?        00:00:00 systemd
    405 ?        00:00:00 (sd-pam)
    427 pts/1    00:00:00 bash
    582 pts/0    00:00:00 proc
    584 pts/0    00:00:00 proc
    585 pts/0    00:00:00 proc
    586 pts/0    00:00:00 ps
Процесс с ID 585 породил процесс с ID 587 (процесс №8)
Процесс №8: ID = 587, ID родителя = 585
Процесс с ID 587 и ID родителя 585 завершает работу
Процесс с ID 585 и ID родителя 584 завершает работу
Процесс с ID 584 породил процесс с ID 588 (процесс №6)
Процесс №6: ID = 588, ID родителя = 584
Процесс с ID 588 и ID родителя 584 завершает работу
Процесс с ID 584 и ID родителя 582 завершает работу
Процесс с ID 582 породил процесс с ID 589 (процесс №4)
Процесс №4: ID = 589, ID родителя = 582
Процесс с ID 589 и ID родителя 582 завершает работу
Процесс с ID 582 и ID родителя 355 завершает работу
```

Генеалогическое дерево процессов:



Exec: Заменяет контекст процесса текущего процесса командой ps -u, которая выводит список процессов конкретного пользователя (или пользователей) с подробной информацией о них, оставляя PID таким же.

Вывод: Познакомились с компилятором GCC и с системными вызовами управления процессами в Linux.